UTN FRM · INGENIERÍA EN SISTEMAS DE INFORMACIÓN · CIENCIA DE DATOS 2026

Guión de la defensa

Entrega 1 · Ingeniería de datos · Veinte minutos, cinco voces, una sola pregunta de fondo: ¿tenemos un pipeline que produce el dataset sin que nadie lo toque a mano?

Grupo 5K09-02

Ezequiel Aguilar · Matías Telo · Anthony Cochachin

Rodrigo Viluron · Facundo Aracena

Proyecto **FF-15** — predicción de la duración de una partida de League of Legends

Exposición: miércoles 9 de septiembre, 14:55 – 15:15

Documento de apoyo: `docs/documentacion-proyecto.pdf`

SECCIÓN 1

Cómo usar este guión

Este documento existe por una sola razón: **la nota es individual y las preguntas van dirigidas**. La consigna lo dice sin vueltas — «que hable uno solo es el error más caro». Así que los doce minutos de exposición están repartidos en cinco turnos de duración casi idéntica, y cada turno tiene el texto escrito de lo que hay que decir.

El guión no se lee en voz alta el día de la entrega. Se ensaya hasta que se puede decir mirando la pantalla, no el papel. Lo que está escrito es el *piso*: el contenido que sí o sí tiene que salir de esa boca en ese minuto. Cómo se dice es de cada uno.

Las cuatro reglas del guión

REGLA	POR QUÉ
Nadie interrumpe el turno de otro	Si a alguien se le escapa un dato, el que sigue lo retoma en su turno. Interrumpir para corregir a un compañero delante del docente hace que el grupo se vea desarmado, y come tiempo que no se recupera.
El turno se cierra con el pase	Cada bloque termina con una frase que nombra al siguiente. Es lo que hace que la exposición se vea coreografiada y no improvisada, y evita el silencio incómodo entre turnos.
El reparto vale para exponer, no para saber	Los doce minutos están repartidos. Las respuestas **no**. En los minutos 12 a 18 el docente elige a quién le pregunta qué, y la pregunta sobre la extracción le puede caer a quien escribió la transformación. La sección 6 es la parte que todos tienen que saber igual.
Si te toca y no sabés, decilo y devolvé algo	«Eso lo trabajó Rodrigo, pero lo que sí te puedo decir es…» y decís lo que sabés. Nunca inventar: un dato inventado se descubre en la repregunta siguiente y arrastra la nota de todo el bloque. Ver la sección 5.

> LA FRASE QUE RESUME LA ENTREGA
>
> Si el docente se lleva una sola idea, que sea ésta: **«El dato crudo está en disco, el pipeline lo reconstruye entero sin tocar la red, y todo lo que decidimos está medido y declarado — incluso lo que nos salió mal.»** Cada uno de los cinco turnos empuja esa idea desde un ángulo distinto.

SECCIÓN 2

Antes de entrar al aula

La consigna es explícita sobre lo que no hay que hacer: llegar sin el entorno levantado y correr el pipeline en vivo. Las dos cosas se comen la mitad de los veinte minutos.

La noche anterior (martes 8)

	QUÉ	QUIÉN
☐	Probar que `astro dev start` levanta sin errores en la notebook que se lleva.	Quien opera
☐	Verificar que la corrida en verde del 6/9 sigue visible en el historial del DAG.	Quien opera
☐	Confirmar que `include/output/lol_2026-09-06.csv` está en disco (11 MB) y abre.	Quien opera
☐	Simulacro completo: los cinco turnos cronometrados, y después los demás preguntan del banco de la sección 8.	Todos
☐	Desactivar suspensión e hibernación en la configuración de energía de Windows.	Quien opera

El mismo día, quince minutos antes

	QUÉ	DETALLE
☐	Regenerar la API key de Riot en el portal y actualizarla en `airflow_settings.yaml`.	Vence cada 24 h
☐	`astro dev restart` y esperar a que la interfaz responda.	Localhost:8080
☐	Dejar **tres pestañas abiertas** en el navegador, en este orden.	Ver abajo
☐	Silenciar notificaciones de Windows y poner el brillo al máximo.	Se proyecta

LAS TRES PESTAÑAS, EN ORDEN

1. Airflow en la vista *Graph* de la corrida en verde del 6/9, con las seis instancias de `land_bronze` desplegadas. **2.** El archivo `include/output/reports/calidad.md`. **3.** El CSV `lol_2026-09-06.csv` abierto (Excel o el visor que sea), con la primera fila visible y las columnas `match_id` y `game_duration_s` a la vista.

La cuarta pestaña opcional, por si preguntan por el bronce: el explorador de archivos parado en `include/output/bronze/matches/plataforma=kr/`.

SI EL DOCENTE PIDE VER UNA CORRIDA EN VIVO

Se dispara con `partidas_objetivo` chico — 10 o 20. Termina en minutos y produce **el mismo CSV completo de 19.145 filas**, porque `refine_silver` reprocesa todo el bronce acumulado, no sólo lo que bajó esa corrida. Es una demostración fuerte y barata: conviene tenerla preparada, pero *además* de la corrida ya hecha, nunca en lugar de ella.

SECCIÓN 3

El reparto de un vistazo

Ezequiel 0:00–2:20	Matías 2:20–4:40	Anthony 4:40–7:00	Rodrigo 7:00–9:20	Facundo 9:20–11:40	Preguntas dirigidas 12:00–18:00	Devolución 18–20

QUIÉN	RELOJ	QUÉ EXPONE	BLOQUE DE LA CONSIGNA
Ezequiel Aguilar	0:00 – 2:20	La pregunta del proyecto, qué es una fila, la fuente, y cómo se llega de un ranking de jugadores a un dataset de partidas.	«De qué se trata»
Matías Telo	2:20 – 4:40	El grafo en verde y las tareas 1 a 4: el sensor, el cortocircuito, el catálogo y el reparto de cuota.	«La corrida»
Anthony Cochachin	4:40 – 7:00	La tarea 5, `land_bronze`: la capa bronce, el filtro previo, el append-only y el pool de keys.	
Rodrigo Viluron	7:00 – 9:20	Las tareas 6 a 9: la transformación sin red, la deduplicación, la validación dura y el reporte de calidad.	«El dataset»
Facundo Aracena	9:20 – 11:40	El CSV abierto: los seis criterios con sus números, la columna objetivo, los nulos, la confesión de tidy y el sesgo declarado.	
11:40 – 12:00		*Colchón. Si se fue de tiempo, se recupera acá. Si sobra, Ezequiel cierra con una frase y abre las preguntas.*	

Cada turno son **2 minutos 20**, salvo el colchón. Son unas 300 palabras habladas a ritmo tranquilo: alcanza para lo que está escrito y para señalar la pantalla, no para improvisar de más. **Cronometrar el simulacro es lo único que garantiza que los cinco entren.**

Además del turno, cada uno tiene un área

El área es de quién se espera que sepa *más* sobre ese tema en las preguntas dirigidas. No es exclusiva: es una prioridad de lectura de la documentación integral.

QUIÉN	ÁREA QUE DOMINA	QUÉ LEER DEL DOCUMENTO INTEGRAL
Ezequiel	La fuente y el proyecto. La API de Riot, los límites, el vencimiento de la key, las dos familias de hosts, los siete endpoints, el recorrido de semilla a dataset. También es quien opera la notebook.	Partes I y III completas · VI
Matías	La orquestación. Qué es un DAG, sensores, cortocircuito, `trigger_rule`, mapeo dinámico, XCom, por qué Airflow y no un script.	Parte II completa · IV.5
Anthony	La extracción y el bronce. `riot.py` y `seeds.py`: el pool de keys, el limitador proactivo, la afinidad key↔puuid, el modelo medallón, el particionado en disco.	III · IV.3 · IV.4.1 · IV.4.3 · V.6
Rodrigo	La transformación. `transform.py` y `schema.py`: los filtros de admisión, la corrección de milisegundos, la asignación de carriles, cómo se generan las 137 columnas, los tests.	IV.4.4 · IV.4.5 · IV.6 · V.2 · V.3
Facundo	El dataset y sus límites. Los seis criterios medibles, las variables derivadas, los nulos imputados, tidy, y las cinco limitaciones declaradas — sobre todo el sesgo por tier.	V completa · VII.2 a VII.5

SECCIÓN 4

El guión, minuto a minuto

Lo que está sobre fondo amarillo es **el texto para decir**. Está escrito para leerse en voz alta y sonar hablado, no escrito. Lo de abajo son los datos que sostienen ese texto, por si el docente frena en el medio.

1 · Ezequiel Aguilar — 0:00 → 2:20

De qué se trata · la pregunta, la fila, la fuente y el recorrido

En pantalla: nada todavía, o Airflow ya abierto de fondo. Este bloque se dice mirando al docente, no a la pantalla. Sin filminas — la consigna lo pide así.

LO QUE DECÍS

Buenas. Somos el grupo 5K09-02. Nuestra pregunta es si se puede predecir cuánto va a durar una partida de *League of Legends* sabiendo únicamente lo que se conoce antes de que arranque: quiénes juegan, con qué campeones y con qué hechizos. La fuente es la API oficial de Riot Games, más Data Dragon, que es el catálogo público del juego.

Una fila de nuestro dataset es **una partida clasificatoria Solo/Duo de la Season 16**, identificada por su `match_id`. Y la columna que queremos predecir es `game_duration_s`: la duración en segundos. Es un número continuo, así que es un problema de regresión.

Hay una cosa de la fuente que conviene decir de entrada porque condiciona todo el diseño: Riot no publica un listado de partidas. Publica un ranking de jugadores. Entonces el camino es entrar por la escalera de un servidor, sacar jugadores semilla, pedirle a cada uno sus últimas quince partidas clasificatorias, y de cada partida bajar el detalle completo más el perfil de los diez que jugaron. Medimos cuánto cuesta eso: **16,8 pedidos a la API por partida**. Uno es la partida, los otros dieciséis son los perfiles.

Y hay dos características de la API que explican por qué el pipeline está armado como está. La *development key* vence cada 24 horas, y los límites de pedidos se aplican **por región, no globalmente**. Lo primero es la razón por la que el DAG se dispara a mano y no está programado. Lo segundo es la razón por la que hay seis tareas de descarga en paralelo, una por servidor.

El resultado de todo eso es un CSV de **19.145 partidas por 137 columnas**. Matías les cuenta cómo arranca el pipeline.

Los datos de este bloque, por si frenan acá:

- Cola 420 = Ranked Solo/Duo. Es la única que recolectamos, para que todas las partidas sean comparables.
- Seis plataformas: `la1` (LAN), `la2` (LAS), `br1`, `na1`, `euw1`, `kr`.
- Los endpoints de partidas viven en hosts continentales (*americas*, *europe*, *asia*); los de ranking, perfil y maestría, en el host de la plataforma.
- Data Dragon es un CDN público: no pide key y no tiene límite de pedidos.

→ Pase: «Matías les cuenta cómo arranca el pipeline.»

2 · Matías Telo `2:20 → 4:40`

La corrida en verde · tareas 1 a 4 del grafo

En pantalla: pestaña 1 — vista *Graph* de la corrida del 6/9, todas las tareas en verde. Señalar cada tarea con el cursor mientras se la nombra.

> LO QUE DECÍS
>
> Ésta es la corrida del 6 de septiembre, completa y en verde. El DAG se llama `lol_ingest` y tiene nueve tareas. Voy con las cuatro primeras.
>
> La primera es `wait_for_riot`, y es un **sensor**: una tarea que espera algo que no controlamos. Comprueba que la API responda y que la key sirva. Sondea cada dos minutos hasta media hora, en modo *reschedule*, que quiere decir que libera el worker entre sondeo y sondeo en vez de ocuparlo durmiendo. Está primero a propósito: la recolección puede durar horas, y enterarnos a la hora seis de que la key venció sería el peor resultado posible.
>
> Y distingue dos fallas que se arreglan distinto. Si Riot está caído, esperar sirve, así que espera. Si ninguna key del pool es válida, esperar no arregla nada, así que corta en el acto. Lo probamos: con una key inválida esta tarea resuelve en **tres segundos** en vez de sondear treinta minutos.
>
> La segunda es `check_key`, el **cortocircuito**. Si el sensor no consiguió respuesta, devuelve falso, todo lo que está abajo queda salteado y la corrida termina bien, no fallada. Tiene una particularidad que vale la pena marcar: corre con `trigger_rule=ALL_DONE`, porque la tarea de arriba puede haber quedado en *skipped*, y con la regla por defecto esta tarea no se ejecutaría nunca justo cuando más falta hace.
>
> La tercera es `fetch_ddragon`: baja el catálogo de campeones y hechizos del CDN público. Se guarda crudo y **particionado por versión**, porque los campeones cambian entre parches y una partida vieja tiene que poder reinterpretarse con el catálogo que le tocaba.
>
> Y la cuarta es `plan_plataformas`, que reparte el objetivo de partidas entre los seis servidores. Se reparte por adelantado porque cada tarea mapeada corre en su propio proceso: no hay un contador compartido en memoria como tendría un script monolítico. Anthony sigue con la tarea que baja los datos.

Los datos de este bloque:

- El grafo: `wait_for_riot → check_key → { fetch_ddragon , plan_plataformas → land_bronze[6] } → refine_silver[6] → consolidate → validate → save`.
- `schedule=None`, `catchup=False`, `max_active_tasks=6`.
- Cuatro parámetros de corrida: `partidas_objetivo`, `plataformas`, `tiers`, `forzar`.
- El sensor usa `soft_fail=True`: agotar el tiempo deja la tarea *salteada*, no *fallada*. Que Riot no responda no es un error de nuestro pipeline.

→ Pase: «Anthony sigue con la tarea que baja los datos.»

3 · Anthony Cochachin — 4:40 → 7:00

`land_bronze` · la capa bronce, el filtro previo y el pool de keys

En pantalla: misma vista *Graph*, con `land_bronze` desplegado en sus seis instancias (aparecen con el nombre de la plataforma: LAS, NA, BR, KR, EUW, LAN). Si hay tiempo, abrir el log de una y mostrar el contador de partidas.

LO QUE DECÍS

Ésta es `land_bronze`, y son seis: una por plataforma, con **mapeo dinámico de tareas**. Es la única tarea de todo el DAG que toca la red. No parsea, no limpia, no valida: guarda el JSON tal como vino de Riot y anota dónde quedó. Eso es la **capa bronce** del modelo medallón.

Hay tres detalles del diseño que vale la pena marcar.

El primero es el **orden de las operaciones**. Bajamos el detalle de la partida y ahí mismo aplicamos los filtros de admisión: que sea Season 16, que dure más de cinco minutos, y que tenga diez participantes, cinco y cinco. Recién si pasa salimos a pedir los perfiles de los diez jugadores. Filtrar antes ahorra unos dieciséis pedidos por cada partida que igual se iba a tirar. Y la partida descartada queda anotada en un índice en disco: **no se vuelve a bajar nunca**, ni en esta corrida ni en las que vengan. Tenemos 1.651 anotadas ahí.

El segundo es que **el bronce es append-only**. Antes de cada pedido se chequea si el archivo ya está en disco; una partida que ya bajamos no se vuelve a pedir jamás. Y la escritura es atómica: se escribe a un temporal y se renombra, así una tarea cortada a la mitad no deja un archivo truncado que reviente el parseo después.

El tercero es el **pool de keys**. La *Connection* de Airflow acepta varias keys separadas por coma, una por integrante, y el cliente reparte los pedidos entre ellas llevando la cuenta por par key-servidor. Acá tuvimos el bug más caro del proyecto: las seis plataformas fallaban con HTTP 400 desde el primer pedido. Resulta que Riot encripta cada identificador de jugador **para la key puntual que lo pidió**, así que si otra key del pool lo usa da error 400, con un mensaje que parece un dato corrupto y no una key equivocada. Lo resolvimos haciendo que cada identificador viaje junto con su key dueña.

Hoy el bronce son **19.145 partidas y 290.654 perfiles: 1,1 GB comprimido** en disco. Rodrigo sigue con la transformación.

Los datos de este bloque:

- Bronce particionado estilo Hive: `bronze/matches/plataforma=kr/KR_1234.json.gz`, `bronze/players/plataforma=kr/puuid=…/`, `bronze/ddragon/version=16.17.1/`.
- Todo comprimido con gzip: sin comprimir pasaría holgadamente los 8 GB. Una partida pesa ~11 KB comprimida.
- Límites de la development key: 20 pedidos por segundo y 100 cada 2 minutos, *por key y por host*.
- El limitador es **proactivo**: calcula si hay cupo antes de disparar, en vez de esperar el 429. El 429 queda como red de contención.
- Ritmo real medido: ~1.154 partidas/hora con las seis plataformas en paralelo. 14 h 39 min para ~17.000 partidas.

→ Pase: «Rodrigo sigue con la transformación.»

4 · Rodrigo Viluron 7:00 → 9:20

De bronce a plata · tareas 6 a 9 y el reporte de calidad

En pantalla: se sigue en el grafo hasta nombrar `save`, y en la última frase se pasa a la **pestaña 2**: `reports/calidad.md`.

LO QUE DECÍS

De acá para abajo el pipeline **no toca la red**: lee del disco.

`refine_silver` también son seis, una por plataforma, y convierte el JSON crudo en filas tipadas. Toda la lógica de transformación es pura: no toca red, no toca Airflow, y se puede testear sola. Eso tiene una consecuencia muy concreta: si mañana descubrimos que veníamos interpretando mal una columna, corregimos el transformador y reprocesamos el 1,1 GB que ya está en disco. Son **trece minutos de reproceso contra las catorce horas y media** que costó bajarlo.

Un ejemplo real de por qué eso importa. Riot cambió el criterio del campo `gameDuration` alrededor del parche 11.20: algunas partidas lo devuelven en segundos y las anteriores en milisegundos. Mezclar las dos unidades en la columna objetivo arruinaría el target *sin que nada falle*, que es la peor clase de bug. Lo normalizamos, y hay un test de regresión que lo cuida.

Acá también encontramos un bug nuestro: la primera versión de esta tarea sólo procesaba las partidas que había bajado esa corrida. Con eso el CSV nunca hubiera crecido entre corridas. Ahora recorre todo el bronce acumulado, y por eso una corrida de dos minutos regenera el dataset entero.

`consolidate` junta las seis plataformas, **deduplica por** `match_id` y castea los tipos. Ésa es la deduplicación que sostiene la promesa de que la clave primaria es única.

`validate` son los chequeos duros: que las columnas sean exactamente las 137 esperadas, que la clave no repita, que no haya nulos donde no puede haberlos y que el objetivo esté en un rango plausible. Si algo de eso falla, **el DAG falla**: no publicamos un dataset roto. Y de paso escribe este reporte de calidad, que es el que trajimos —acá están los seis números que pide la consigna, calculados por la corrida, no a mano.

Y `save` escribe el CSV fechado: `lol_2026-09-06.csv`. Facu abre el archivo.

Los datos de este bloque:

- Los cuatro filtros de admisión: `gameVersion` empieza con «16.», duración ≥ 300 s, exactamente 10 participantes, 5 y 5 por equipo.
- Las 137 columnas se *generan*, no se escriben a mano: 3 + (10 variables × 5 posiciones × 2 equipos) + (17 × 2).
- 10 tests en verde, `ruff` sin hallazgos, `astro dev parse` limpio.
- Dos de esos tests son de regresión sobre bugs reales: la duración en milisegundos y el `teamPosition` vacío que alimenta `roles_inferidos`.
- `save` toma la fecha de `dag_run.logical_date or dag_run.run_after`, nunca de `ds`: `ds` sólo existe si el DAG tiene *schedule*, y el nuestro corre a demanda.

→ Pase: «Facu abre el archivo.»

5 · Facundo Aracena `9:20 → 11:40`

El dataset · los seis criterios, los nulos, tidy y el sesgo

En pantalla: pestaña 3 — el CSV abierto. Señalar la columna `match_id` al hablar de la clave y `game_duration_s` al hablar del objetivo. Volver a la pestaña 2 (`calidad.md`) para los números si es más cómodo leerlos de ahí.

LO QUE DECÍS

Éste es el CSV que produjo esa corrida. Voy por los criterios.

Clave sin duplicados: verdadero. La clave es `match_id`, viene de la fuente —no la inventamos nosotros— y ya trae el prefijo del servidor donde se jugó, así que es única a nivel global sin combinarla con nada. **Volumen: 19.145 filas**, diecinueve veces el mínimo de mil. **Ancho: 137 columnas. Mezcla de tipos:** 106 enteras, 20 flotantes y 11 de texto. **Columnas vacías: ninguna.**

La columna objetivo es `game_duration_s`: media 1.748 segundos —29 minutos—, mediana 1.751, o sea prácticamente simétrica, y rango de 301 a 3.963 segundos. El piso de 301 no es casualidad: es el filtro que descarta los remakes.

Nulos: no hay ninguno, y ésta es la parte que quiero explicar bien, porque no es que los datos vengan perfectos. Donde la fuente no tiene el dato, imputamos un valor que *significa* algo: un jugador sin ranking queda con elo 1500 y winrate 0,50, que codifica «habilidad desconocida, asumimos promedio»; un jugador que nunca usó ese campeón queda con cero puntos de maestría, que es literalmente cierto. Y la contra la decimos nosotros: imputar 1500 mezcla «es de nivel medio» con «no sabemos nada de él». La alternativa sería una columna `es_unranked` explícita —el código ya la calcula, sólo no la escribe— y la dejamos deliberadamente abierta para la Unidad 2.

Y dos cosas que preferimos decir antes de que nos las pregunten.

Una: **el dataset no es del todo *tidy***. La observación más fina que existe acá es jugador-en-partida, y hay diez por partida; están aplanados en cien columnas en lugar de diez filas. Es una concesión deliberada: la unidad de nuestra pregunta es la partida, y en formato largo el target se repetiría idéntico diez veces, que rompe la correspondencia una-fila-una-observación justo respecto de lo que queremos predecir.

Dos: tenemos un **sesgo de muestreo por tier**, y lo tenemos medido. Cada servidor quedó con partidas de un rango casi puro y distinto: el elo medio va de 1.627 en Norteamérica a 3.231 en LAN. Sabemos por qué pasa, sabemos qué implica y sabemos cómo se corrige.

Los datos de este bloque:

- Las 137 columnas: 3 de identidad/objetivo/calidad + 100 por jugador + 34 de agregados de equipo.
- Tres columnas derivadas: `player_elo`, `is_main` (umbral 50.000 puntos) y `roles_inferidos`.
- Sólo 17 filas de 19.145 tienen `roles_inferidos > 0`. Se filtran con `df[df.roles_inferidos == 0]`.
- Cuartiles del objetivo: Q1 = 1.496 s, mediana = 1.751 s, Q3 = 2.010 s.

- Correlación elo↔duración: −0,16 global, −0,25 en KR (la única con rango amplio), −0,01 en NA1. La señal aparece donde hay variación: es la firma del sesgo.

→ Cierre (Ezequiel, si sobra tiempo): «Eso es lo que trajimos. Quedamos para las preguntas.»

SECCIÓN 5

Reglas de la mesa · los minutos 12 a 18

Acá se juega la nota. El docente elige a quién le pregunta qué, y explícitamente avisó que «la pregunta sobre el *scraping* puede caerle a quien escribió la transformación». Estas cinco reglas valen para los cinco.

SITUACIÓN	QUÉ HACER
Te preguntan de tu área	Respuesta corta primero —una o dos oraciones que contesten exactamente lo que preguntaron— y después el detalle. No arrancar por el contexto: arrancar por la respuesta.
Te preguntan de otra área y la sabés	Contestala. No mires al compañero ni le pases la pelota: que la pregunta le haya tocado a otro es *a propósito*. Mirar al que «le corresponde» es exactamente lo que el docente está buscando detectar.
Te preguntan algo que sabés a medias	Decí lo que sí sabés y marcá el límite: «El detalle exacto del limitador lo trabajó Anthony; lo que sí te puedo decir es que el control es proactivo, calcula el cupo antes de mandar el pedido». Media respuesta honesta suma; una respuesta inventada resta el doble.
No lo sabés	«Eso no lo sé» + dónde está la respuesta: «está en la parte IV de nuestra documentación, lo trabajó Rodrigo». Es una respuesta aceptable *una vez*. La segunda ya no.
Te preguntan por un problema o una limitación	No lo minimices. Nombralo, decí cómo lo detectaste, qué hiciste y qué queda pendiente. Las cinco limitaciones de la sección 7 son puntos a favor cuando se cuentan enteras y en contra cuando se esquivan.

LO QUE NO HAY QUE DECIR

- **«No hay nulos»** a secas. Siempre seguido de *«porque imputamos, y esto es lo que imputamos y por qué»*.
- **«El dataset es representativo»**. No lo es, y está medido que no lo es. Decirlo sería el peor error posible.
- **«Corre solo todos los días»**. No: `schedule=None`, se dispara a mano, y hay una razón concreta.
- **«El elo lo sacamos de Riot»**. Riot no publica el MMR. `player_elo` es una construcción nuestra.
- **«Eso lo hizo la IA»** o **«no sé, eso lo hizo otro»** como respuesta completa. Ver la pregunta 26.
- **Números redondeados de memoria.** Si no te acordás el número exacto, decí el orden de magnitud y señalá el reporte: está proyectado en la pantalla.

SECCIÓN 6

Lo que todos tienen que saber igual

Esto **no se reparte**. Son las cinco preguntas que la consigna dice textualmente que se van a preguntar, más los seis números. Los cinco integrantes las tienen que poder responder de memoria.

Las cinco preguntas de la consigna

1 · «Una fila es un ___»

Una partida clasificatoria de League of Legends, modalidad Solo/Duo, de la Season 16.

Y coincide con la pregunta del proyecto: queremos predecir cuánto dura *una partida*, y la unidad sobre la que sacamos conclusiones es exactamente ésa. Si la unidad fuera el jugador, habría un desajuste con la pregunta.

2 · ¿Cuál es la columna objetivo?

`game_duration_s`: la duración de la partida en segundos. Es continua, así que es un problema de regresión.

Sale de `info.gameDuration` del detalle de partida, con la corrección de milisegundos. Como estructura latente alternativa, el dataset también permite agrupamiento no supervisado sobre las columnas de composición, para encontrar arquetipos de equipo.

3 · ¿Por qué hay nulos donde hay nulos?

No hay nulos — y hay que explicar por qué, porque no es que los datos vengan perfectos. Donde la fuente no tiene el dato, imputamos un valor que significa algo.

Sin ranking: `player_elo = 1500` y `player_winrate = 0.50` («habilidad desconocida, asumimos promedio»; 1500 cae entre Gold y Platinum, la zona más poblada). **Nunca jugó ese campeón:** `champ_mastery_pts = 0` — no es imputación, es el valor real. **Sin `perks` en el payload:** runas en 0, que no es un identificador válido, así que el caso queda distinguible. **La contra:** imputar 1500 mezcla «es de nivel medio» con «no sabemos nada de él»; la alternativa es una columna `es_unranked`, pendiente para la Unidad 2.

4 · ¿Qué pasa si lo corrés de nuevo? ¿Sale el mismo archivo?

No: sale uno más grande, y es a propósito. Hay que separar dos cosas.

De `refine_silver` para abajo el pipeline es una **función pura del disco**: con el mismo bronce produce siempre exactamente el mismo CSV. Pero `land_bronze` es **acumulativo**: cada corrida busca partidas que todavía no tiene; las que ya están nunca se vuelven a pedir, las nuevas se suman. Es la única forma de juntar volumen con una API que permite 100 pedidos cada 2 minutos por región y una key que vence cada 24 horas. Si quisiéramos congelar el dataset, alcanza con no volver a correr la descarga y reprocesar el bronce tal como está.

5 · ¿Dónde guardás el dato crudo? ¿Tenés el bronce?

Sí, y guardamos más que el mínimo: en `include/output/bronze/` queda la respuesta JSON de la API tal como llegó, comprimida y particionada por plataforma.

No guardamos sólo las partidas: guardamos **las tres respuestas que hacen falta para reconstruir una fila** —el detalle de la partida, la entrada de ranking de cada jugador y su maestría— más el catálogo de campeones. Por eso la capa plata se reconstruye 100 % sin red. La prueba concreta: si mañana descubrimos que veníamos interpretando mal una columna, corregimos `transform.py` y reprocesamos los 1,1 GB que ya están en disco, sin gastar un solo pedido a Riot. Son ~13 minutos contra las 14 h 39 min que costó bajarlos.

Los seis números

CRITERIO	CÓMO SE VERIFICA	RESULTADO (6/9/2026)
Clave sin duplicados	`df["match_id"].is_unique`	**True**
Volumen suficiente (> 1.000)	`len(df)`	**19.145**
Ancho suficiente (≥ 5)	`df.shape`	**137 columnas**
Mezcla de tipos	`df.dtypes.value_counts()`	**106 int · 20 float · 11 object**
Nulos conocidos	`df.isna().mean()`	**ninguno**
Sin columnas vacías	`df.columns[df.isna().all()]`	**vacío**

EL CRITERIO QUE MÁS PESA

La consigna lo dice: la clave primaria «es el test operativo de la unidad de análisis: si la clave repite, o la unidad está mal definida o el pipeline está duplicando filas». **Nuestra respuesta tiene tres capas:** el `match_id` viene de la fuente y ya incluye el prefijo de la plataforma, así que es único globalmente; `consolidate` hace un `drop_duplicates("match_id")` explícito al juntar los seis parciales; y `validate` hace fallar el DAG si aun así llegara a repetirse.

SECCIÓN 7

Los seis tropiezos, y cómo los resolvimos

Esto es material de oro para las preguntas dirigidas. Un grupo que cuenta un bug real, cómo lo detectó y qué cambió después demuestra que entendió el pipeline; uno que dice «funcionó todo bien» demuestra que lo corrió una vez y no lo miró. **Cada integrante debería poder contar al menos tres de estos seis.**

1 · El HTTP 400 que parecía un dato corrupto

ANTHONY

Síntoma: con el pool de siete keys, las seis tareas `land_bronze` fallaban con `400 Bad Request — Exception decrypting <puuid>` desde el primer pedido, con todas las keys por igual. El mensaje sugiere un dato corrupto.

Causa: Riot devuelve cada `puuid` *encriptado para la key puntual que hizo esa request*. Cualquier otra key del pool, aunque sea válida, no lo puede leer.

Cómo se confirmó: pegándole a la misma URL con distintas keys: una respondía 200 y las demás 400.

Solución: afinidad key↔puuid. Cada `puuid` viaja con su «key dueña» desde `iter_puuids` hasta el pedido de perfil, y el limitador acepta un parámetro para no rotar. Si esa key venció en el medio, se saltea el pedido en vez de gastar reintentos contra keys que nunca lo van a poder leer.

2 · La hibernación de las 03:19

MATÍAS / EZEQUIEL

Qué pasó: el 6/9 la notebook entró en hibernación con cuatro tareas `land_bronze` a mitad de camino, entre 2.520 y 2.610 partidas de 3.334. Los procesos murieron en seco — los logs se cortan sin *traceback*.

Qué no pasó: no se perdió un solo archivo del bronce. A las 07:16 el *scheduler* detectó las tareas huérfanas y las relanzó solo, sin que nadie tocara nada. El dataset se generó igual ese mismo día.

Por qué importa: es la mejor defensa de la arquitectura que tenemos. El estado vive en disco, no en memoria ni en XCom. Con un script monolítico habría que haber empezado de cero. *Lección operativa:* desactivar suspensión antes de una corrida larga.

3 · El CSV que nunca hubiera crecido

RODRIGO

Bug: la primera versión de `refine_silver` iteraba sólo las partidas que había bajado *esa* corrida. Cada corrida producía un CSV con su propia cuota y pisaba el anterior: toda la estrategia de acumular hacia 20.000 filas en varias tandas no funcionaba.

Solución: `glob` sobre todo el bronce de la plataforma.

Verificación a escala real (6/9): con 19.133 partidas en bronce, una corrida de cuota 2 por plataforma bajó 12 partidas nuevas y produjo un CSV de **19.145 filas** — el acumulado entero, no las 12 de esa corrida. Es lo que permite regenerar el entregable en 15 minutos sin volver a recolectar.

4 · El rol asignado en silencio

RODRIGO

Bug heredado del prototipo: cuando la fuente no informaba `teamPosition` —pasa en remakes y con *autofill*— el jugador se colocaba en el slot que sobraba y quedaba etiquetado como jungla sin haberlo sido. El dato estaba mal y **no había forma de saberlo**.

Solución: la columna `roles_inferidos` cuenta cuántos de los 10 quedaron sin emparejar. El error sigue existiendo —la fuente no tiene el dato— pero ahora es visible y esas filas se filtran. Son 17 de 19.145.

Cómo se dice esto: «no arreglamos el dato, lo hicimos visible». Es una distinción que vale.

5 · La columna que valía siempre 1

RODRIGO / FACUNDO

El esquema del prototipo traía `t1_count_smite` y `t2_count_smite`. En toda la muestra valían 1: el juego le impone Smite al jungla y hay un jungla por equipo. Una columna de **varianza cero** no aporta nada a un modelo, así que se quitó. Los otros tres conteos de hechizo —flash, teleport, ignite— sí varían y se conservan.

La cuenta de las columnas: 138 del prototipo + 1 (`roles_inferidos`) − 2 (los dos *smite*) = **137**.

6 · La extrapolación que nos hizo planificar mal

EZEQUIEL

Una versión anterior de nuestra documentación proyectaba ~6.400 partidas/hora con 6 keys, extrapolando linealmente desde una. La corrida real dio **1.154/hora con 7 keys**: cinco veces y media menos.

Por qué: el techo no lo pone el cupo de las keys sino la concurrencia del cliente. Dentro de cada tarea las partidas se bajan una por una y sólo los perfiles van en paralelo, con cuatro hilos. Sumar keys agranda un cupo que no se está llegando a consumir.

Qué cambiamos: planificamos con el número medido, no con el proyectado, y pasamos a una estrategia por tandas. Si en algún momento hace falta acelerar de verdad, el cambio que rinde es subir la concurrencia, no juntar más keys.

SECCIÓN 8

Banco de preguntas probables

Treinta y dos preguntas, agrupadas por tema. La etiqueta gris de la derecha dice quién es el *dueño natural* de esa respuesta — pero el docente pregunta a quien quiere, así que la etiqueta sirve para saber quién la prepara **a fondo**, no para que los demás la salteen.

A · La pregunta, la unidad de análisis y el dataset

1 · ¿Por qué la duración y no quién gana?

EZEQUIEL

Porque la duración se puede predecir desde condiciones iniciales sin que el resultado contamine nada, y es una variable continua, que nos da un problema de regresión.

Predecir el ganador desde el draft es un problema muy estudiado y con techo bajo — el matchmaking está justamente diseñado para que sea 50/50. La duración, en cambio, depende de la composición de los equipos de una forma más directa: un equipo de tanques y uno de asesinos no juegan partidas de la misma longitud. Es una hipótesis que vamos a testear en la Unidad 2.

2 · ¿Por qué 137 columnas? ¿No es demasiado ancho?

FACUNDO

Porque una fila describe diez jugadores y dos equipos. Son 3 columnas de identidad, objetivo y calidad, 100 por jugador (10 variables × 5 posiciones × 2 equipos) y 34 de agregados de equipo (17 × 2).

Ninguna es de relleno: las 100 de jugador son las mismas diez variables repetidas por slot, y las 34 de equipo son agregaciones de esas mismas más el catálogo de campeones. Y el ancho es *generado*, no escrito a mano: `schema.py` arma los nombres a partir de dos listas, así que es imposible que se escape un typo y agregar una variable por jugador es cambiar una lista de diez elementos.

3 · ¿El dataset es tidy?

FACUNDO

No del todo, y es deliberado. La observación más fina es jugador-en-partida —hay diez por partida— y están aplanados en 100 columnas en vez de diez filas.

Tres razones. **Una:** la unidad de nuestra pregunta es la partida; en formato largo el target `game_duration_s` se repetiría idéntico en las diez filas, que rompe «una fila, una observación» justo respecto de lo que importa. **Dos:** obligaría a re-agregar antes de cada modelo, duplicando esa lógica en cada análisis en vez de resolverla una vez en el pipeline. **Tres:** es el mismo tipo de concesión que hace el dataset de la cátedra con `player_positions`. Lo que sí cumplimos: ninguna celda guarda listas ni valores múltiples, que es justamente el problema del dataset FIFA.

4 · ¿Qué es `t1_num_tanks` y por qué las seis columnas de tipo no suman 5?

FACUNDO

Cuenta cuántos de los cinco campeones del equipo están etiquetados como *Tank* en Data Dragon. No suman 5 porque un campeón puede tener varias etiquetas a la vez: Ornn es Tank y Fighter.

No es un error de cálculo, es cómo clasifica el catálogo oficial de Riot. Saberlo evita interpretar mal esas columnas en la exploración de la Unidad 2.

5 · ¿Está balanceado el objetivo?

FACUNDO

Es una variable continua, así que la pregunta no es de balance sino de distribución — y está centrada: media 1.748 s, mediana 1.751 s, desvío 424 s.

Media y mediana prácticamente iguales quiere decir que no hay asimetría fuerte. El rango va de 301 a 3.963 segundos, sin colas absurdas. Y como *sanity check* externo: una partida promedio de LoL ronda la media hora, así que nuestros 29 minutos coinciden con lo que se sabe del juego — es una señal de que el parseo no está roto.

6 · ¿Por qué el mínimo es 301 segundos y no 300?

RODRIGO

El filtro admite desde 300 s inclusive; 301 es simplemente la partida más corta que quedó. No hubo ninguna de exactamente 300.

El umbral de 300 s descarta los *remakes*: partidas que se cancelan en los primeros minutos porque alguien no se conectó. No son partidas reales y su duración no tiene relación con las variables que medimos.

B · Airflow y la orquestación

7 · ¿Por qué Airflow y no un script de Python?

MATÍAS

Porque la corrida dura horas y necesita sobrevivir a interrupciones. Airflow nos da reintentos por tarea, logs separados, paralelismo entre las seis plataformas, estado visible y la posibilidad de retomar sin repetir lo hecho.

Y lo probamos sin querer: el 6/9 la máquina hibernó a las 03:19 con cuatro tareas a mitad de camino, y el *scheduler* las relanzó solo a las 07:16 sin que nadie tocara nada. Con un script monolítico habría que haber empezado de cero. (Tropiezo 2 de la sección 7.)

8 · ¿Qué es un DAG y por qué tiene que ser acíclico?

MATÍAS

Un grafo dirigido acíclico: las tareas y las flechas de dependencia entre ellas. Acíclico porque si hubiera un ciclo no existiría un orden de ejecución posible — A esperaría a B y B a A.

El DAG define *qué depende de qué*, no *cuándo*: el «cuándo» lo pone el `schedule`, que en el nuestro es `None`. Nuestro DAG se llama `lol_ingest` y tiene nueve tareas, dos de las cuales están mapeadas dinámicamente sobre las seis plataformas.

9 · ¿Por qué el DAG no está programado?

MATÍAS

Porque la development key de Riot vence cada 24 horas. Un DAG programado corriendo sin nadie mirando fallaría todos los días apenas venciera la key, y el ruido de esas fallas taparía las reales.

Se dispara a mano con la key recién renovada. Es una decisión, no una omisión: el DAG de ejemplo de la cátedra, `pokemon_api`, también usa `schedule=None`. Y va con `catchup=False`, así que si el entorno estuvo apagado una semana no intenta recuperar corridas perdidas.

10 · ¿Qué es un sensor y en qué se diferencia de una tarea normal?

MATÍAS

Una tarea que espera a que se cumpla una condición externa antes de dejar seguir al resto. La nuestra es `wait_for_riot`: sondea la API cada 2 minutos hasta 30.

Dos detalles del nuestro. Corre en modo **reschedule**: entre sondeo y sondeo libera el worker en vez de ocuparlo durmiendo. Y usa `soft_fail=True`, así que agotar el tiempo lo deja en *salteado* y no en *fallado* — que Riot no responda no es un error de nuestro pipeline.

11 · ¿Qué es `trigger_rule` y por qué `check_key` usa `ALL_DONE`?

MATÍAS

Es la condición que decide si una tarea se ejecuta según cómo terminaron sus predecesoras. Por defecto es `ALL_SUCCESS`: sólo corre si todas salieron bien.

`check_key` tiene que correr *justamente* cuando el sensor quedó salteado, que es el caso que tiene que manejar. Con la regla por defecto no se ejecutaría nunca cuando más falta hace. Con `ALL_DONE` corre pase lo que pase con la tarea de arriba, mira el resultado y decide si cortocircuita el resto del DAG.

12 · ¿Qué es XCom y qué mandan por ahí?

MATÍAS

Es el mecanismo de Airflow para pasar valores entre tareas. Por ahí mandamos rutas y metadatos, nunca datos: los datos van a disco.

XCom se guarda en la base de metadatos de Airflow; meter ahí un DataFrame de 19.000 filas la haría explotar y además rompería el modelo medallón, donde cada capa es un archivo en disco que se puede inspeccionar. Lo que viaja es «la plataforma `kr` dejó su parcial en tal ruta, con tantas filas».

13 · ¿Por qué seis tareas de descarga en vez de una?

ANTHONY

Porque Riot aplica los límites por región: cada plataforma tiene su propio cupo. Seis tareas en paralelo multiplican la velocidad en los endpoints de perfil y maestría, que son el cuello de botella real.

Medimos 16,8 pedidos por partida, de los cuales ~16 son perfiles. Con una sola tarea estaríamos usando un solo cupo. La sutileza a tener a mano: `la1`, `la2`, `br1` y `na1` *comparten* el cupo de partidas, porque las cuatro enrutan a *americas*; pero como las partidas son ~1 pedido y los perfiles ~16, en la práctica no molesta.

14 · ¿Por qué no usaron un operador prefabricado para los pedidos HTTP?

MATÍAS / ANTHONY

Porque nuestro cliente necesita rotar entre varias API keys y llevar la cuenta de los pedidos por par (key, host) para respetar los límites. Ningún operador prefabricado hace eso.

La regla de la cátedra: operador cuando la acción es estándar, `@task` propio cuando se necesita lógica específica. Sí usamos la infraestructura de *Connections* de Airflow para la credencial, que es la parte que sí conviene reusar.

15 · ¿Por qué `include/lol/` no importa Airflow?

RODRIGO

Para que la lógica del dominio se pueda testear y ejecutar sin levantar un orquestador. `dags/` tiene el flujo; `include/`, la lógica pesada.

Es lo que permite que `transform.py` sea un módulo puro que recibe un dict y devuelve un dict, y que los tests corran en segundos. También es lo que haría barato cambiar de orquestador algún día: habría que reescribir el DAG, no el proyecto.

C · La fuente, las credenciales y la extracción

16 · ¿Dónde está la API key?

ANTHONY / EZEQUIEL

En una *Connection* de Airflow llamada `riot_api`, declarada en `airflow_settings.yaml`, que está en el `.gitignore`. En el repositorio se versiona `airflow_settings.example.yaml` con un placeholder. La key no aparece en ningún `.py`.

El campo acepta varias keys separadas por coma, una por integrante, y el cliente arma un pool. Hay una sola función en todo el proyecto que toca la credencial, y lee de la Connection. **Si repregunta por qué el proyecto de la cátedra sí versiona ese archivo:** porque su única conexión es la PokéAPI, que es pública y no tiene ningún secreto. En el nuestro hay una credencial real, así que la decisión se invierte.

17 · ¿Cómo eligieron las partidas? ¿La muestra es representativa?

FACUNDO / EZEQUIEL

No es representativa, y lo sabemos medido. Es una muestra de conveniencia sesgada por tier: lo medimos, lo declaramos, y sabemos cómo se corrige.

Por qué pasa: Riot no publica un listado de partidas, publica un ranking. Así que el camino es escalera → jugador semilla → sus últimas 15 partidas. El recorrido mezcla el orden de los tiers justamente para evitar el sesgo, pero el *shuffle* no alcanza: una página de escalera trae 205 jugadores y cada uno rinde hasta 15 partidas, así que una sola combinación tier/división llena la cuota entera de la corrida y nunca se llega al segundo tier. **El resultado:** elo medio de 1.627 en NA1 (33 % Silver, 29 % Bronze) a 3.231 en LA1 (39 % Grandmaster). **Cómo se corrige:** rotar de tier cada N jugadores dentro de la corrida, en vez de confiar en el orden mezclado. El bronce ya bajado se conserva, así que sumar tiers faltantes no obliga a rehacer nada.

18 · ¿Qué implica ese sesgo para el modelo?

FACUNDO

Dos cosas concretas. El poder predictivo del elo está *subestimado*, y plataforma y elo están *confundidos*.

Lo primero: dentro de cada plataforma el elo casi no varía, así que el modelo tiene poco de dónde aprender. La correlación elo↔duración es −0,16 global, pero −0,25 en KR —la única con rango amplio— y apenas −0,01 en NA1. La señal aparece donde hay variación y desaparece donde no la hay: es la firma del sesgo, no una propiedad del juego. **Lo segundo:** en NA las partidas duran 1.826 s en promedio y en BR 1.687 s, pero eso no es un efecto de la región — es que NA quedó lleno de plata/bronce y BR de diamante. Atribuirlo al servidor sería una conclusión falsa, y es justo el error que la materia señala como «el modelo aprende el sesgo de recolección en vez de la señal».

19 · ¿Por qué las últimas 15 partidas de cada jugador y no todas?

EZEQUIEL

Por dos razones: recencia y reparto de cuota.

Recencia: consultamos el winrate y la maestría *hoy*, así que cuanto más reciente la partida, menos desfasado está ese dato — acota la fuga temporal que declaramos. **Cuota:** si a un jugador le pidiéramos su historial entero, un puñado de jugadores se comería toda la cuota de la corrida y el dataset tendría menos diversidad de jugadores.

20 · ¿Por qué sólo la cola 420?

EZEQUIEL

Para que todas las partidas sean comparables entre sí. Mezclar modalidades rompería la comparabilidad.

Una partida de ARAM se juega en otro mapa, con otras reglas y otra duración típica. Si el dataset mezclara modalidades, el modelo estaría aprendiendo a distinguir modalidades en vez de a predecir duración. Restringir a una sola modalidad es una decisión de diseño del dataset.

21 · ¿Qué pasa si Riot está caído cuando corren el pipeline?

MATÍAS

El sensor sondea cada 2 minutos durante media hora antes de rendirse. Si se agota, queda salteado —no fallado—, `check_key` corta y la corrida termina bien sin hacer nada.

Y distinguimos ese caso del de una key inválida: ahí no esperamos, cortamos en el acto, porque esperar no arregla una key vencida. Es la diferencia entre una falla transitoria y una permanente, y el pipeline la trata distinto a propósito.

22 · ¿Y si les piden hacer *scraping*? ¿Por qué API y no scraping?

CUALQUIERA

Nuestra fuente es una API REST oficial, que la consigna acepta explícitamente en el criterio de «descargable de forma automatizada».

Y tiene una ventaja concreta sobre el scraping: la fuente es estable, está documentada y no se rompe cuando alguien cambia el HTML de una página. A cambio pagamos dos costos que el scraping no tiene: una credencial que hay que gestionar y que vence cada 24 h, y límites de pedidos que condicionan toda la arquitectura del pipeline.

D · Reproducibilidad, bronce e idempotencia

23 · Si les cambio el parseo de una columna, ¿tienen que volver a descargar todo?

RODRIGO

No, y ésa es exactamente la razón de tener capa bronce. Se corrige `transform.py`, se vuelve a disparar el DAG y `refine_silver` reconstruye la plata leyendo del disco, sin tocar la red.

Son ~13 minutos de reproceso sobre 19.145 partidas, contra las 14 h 39 min que costó bajarlas. Con 20.000 partidas eso son horas de descarga ahorradas cada vez que cambia un criterio.

24 · ¿Qué pasa si dos jugadores semilla mencionan la misma partida?

ANTHONY

No se baja dos veces: `land_bronze` chequea si el archivo ya existe en disco antes de cada pedido. Y si aun así llegara duplicada, `consolidate` deduplica por `match_id`.

Esto pasa todo el tiempo, y es deseable: es lo que hace que el costo por partida baje a medida que el dataset crece. La plataforma que más lejos llegó terminó en 11,7 pedidos por partida contra 19,4 de la que menos avanzó, porque los perfiles ya guardados salen de caché en vez de la API.

25 · ¿Y si me lo corrés ahora, cuánto tarda?

EZEQUIEL

Con el bronce ya en disco, el DAG completo son 15 minutos de punta a punta. Con `partidas_objetivo` chico —10 o 20—, minutos, y produce el mismo CSV de 19.145 filas porque `refine_silver` reprocesa todo el acumulado.

Lo que *no* es rápido es la recolección: ~1.150 partidas/hora con las seis plataformas en paralelo. Llegar a las 19.145 llevó 14 h 39 min de reloj repartidas en tandas.

26 · ¿Usaron IA para esto?

CUALQUIERA

Sí, con asistencia de IA para escribir y documentar, con reglas escritas en el propio repositorio. Cada decisión de diseño está documentada con su porqué, y la podemos defender — que es justamente lo que estamos haciendo ahora.

Concretamente: el repo tiene un `CLAUDE.md` con las reglas de trabajo (credenciales en Connection, XCom sin datos, nada de inventar requisitos) y una *skill* versionada con las convenciones del proyecto y las trampas de la API. Los bugs que contamos en la sección de tropiezos los encontramos corriendo el pipeline contra la API real, no leyendo código. **La respuesta honesta y corta si insiste:** «la usamos como herramienta; el criterio, las mediciones y las decisiones son nuestras, y por eso las podemos explicar».

27 · ¿Le puedo pasar el repo a otro grupo y les corre?

EZEQUIEL

Sí: clonar, copiar `airflow_settings.example.yaml` a `airflow_settings.yaml`, poner su propia key de Riot y `astro dev start`.

Lo que *no* viaja es el bronce: son 1,1 GB y está en el `.gitignore`, porque son datos, no código. O sea que arrancarían el bronce de cero y tendrían que recolectar. El CSV entregable tampoco se versiona por la misma razón.

E · Preguntas cruzadas y de fondo

28 · Explicame el grafo sin mirar la pantalla.

CUALQUIERA

Sensor → cortocircuito → dos ramas: el catálogo por un lado y el reparto de cuota por otro. El reparto abre seis descargas en paralelo. Las dos ramas se juntan en seis transformaciones, que confluyen en consolidar, validar y guardar.

Nueve tareas: `wait_for_riot`, `check_key`, `fetch_ddragon`, `plan_plataformas`, `land_bronze` (×6), `refine_silver` (×6), `consolidate`, `validate`, `save`. **Practicar esto en el simulacro:** es la pregunta que más rápido distingue a quien entendió el pipeline de quien memorizó la pantalla.

29 · ¿Cómo saben que no hay fuga de información?

FACUNDO / RODRIGO

Porque el esquema excluye por diseño todo lo que ocurre durante la partida: no hay kills, ni oro, ni torres, ni deltas por minuto, ni bans. Sólo entran datos del draft —campeones, hechizos, runas— y perfiles de jugador.

Dicho eso, hay una fuga acotada que declaramos nosotros: `player_winrate` y `champ_mastery_pts` se consultan en el momento de la recolección, no en el momento en que se jugó la partida. Para una partida de hace semanas son valores posteriores al hecho. La API de Riot no expone winrate ni maestría históricos, así que no se puede resolver con esta fuente; lo que sí hacemos es acotar el efecto pidiendo las últimas 15 partidas de cada jugador.

30 · El elo lo construyeron ustedes. ¿No es inventar un dato?

FACUNDO

Es una variable derivada, y la presentamos como tal. Riot no publica el MMR: es privado. Lo que sí publica es el rango visible —tier, división y puntos— y nuestra fórmula convierte esa escala ordinal en un número comparable.

La fórmula es `base_del_tier + bonus_de_división + puntos × 0,5`: 300 puntos entre tiers consecutivos, repartidos en cuatro divisiones de 75, y los puntos de liga pesan la mitad para que un jugador a punto de ascender quede cerca del tier siguiente sin superponerse. Un Oro I con 40 puntos da 1.700 + 225 + 20 = 1.945. Los coeficientes son una elección del grupo, no una medición — y si en la Unidad 3 no aporta, se reemplaza por el tier como categórica sin costo de API, porque el bronce guarda la respuesta cruda.

31 · ¿Qué variable creen que va a explicar más la duración?

CUALQUIERA

Es una hipótesis para la Unidad 2, no un resultado. Apostamos a la composición de equipo antes que al elo.

El razonamiento: un equipo cargado de tanques tiende a partidas largas de desgaste y uno de asesinos a partidas cortas, así que `num_tanks`, `num_assassins` y `total_base_hp` son las candidatas. Del elo esperamos poco *en este dataset*, y no porque el elo no importe sino por el sesgo que ya medimos: casi no varía dentro de cada plataforma. **La forma correcta de contestar esto es marcando que es una hipótesis** — afirmarlo como hallazgo sería inventar un resultado que todavía no medimos.

32 · ¿Qué es la capa oro y por qué no la tienen?

MATÍAS / FACUNDO

Es la capa de negocio del modelo medallón: agregados y variables construidas para un uso puntual. No la tenemos porque la Entrega 1 pide el pipeline y la capa plata; el oro aparece en la Unidad 3, sobre esta misma plata.

Bronce = crudo, tal como llegó, inmutable. Plata = limpio, tipado y validado; es lo que entregamos. Oro = listo para consumir por un modelo o una app. Tener bronce es lo que permite que el oro se pueda rehacer las veces que haga falta.

SECCIÓN 9

Ficha de bolsillo

Una página para imprimir y tener en la mano. Si hay un solo papel sobre la mesa el día de la entrega, que sea éste.

Los números que no se pueden equivocar

Filas del dataset	**19.145**
Columnas	**137** (3 + 100 + 34)
Clave primaria	`match_id` — **única: True**
Columna objetivo	`game_duration_s` — **regresión**
Objetivo: media / mediana / desvío	**1.748 s / 1.751 s / 424 s**
Objetivo: mínimo / máximo	**301 s / 3.963 s**
Tipos	**106 int · 20 float · 11 object**
Nulos	**ninguno, en ninguna columna**
Columnas 100 % vacías	**ninguna**
Filas con rol inferido	**17 de 19.145**
Peso del CSV	**11 MB**

El bronce

Partidas crudas	**19.145**
Perfiles de jugador	**290.654**
Partidas descartadas (índice)	**1.651**
Tamaño en disco	**1,1 GB comprimido** (> 8 GB sin comprimir)
Por plataforma	**kr 3.430 · br1 3.356 · la1 3.355 · la2 3.066 · na1 2.988 · euw1 2.950**

La corrida

Recolección grande	**5/9 11:53 → 6/9 09:33 (≈21 h 40)**
Tiempo neto de descarga	**14 h 39 min para ~17.000 partidas**
Ritmo agregado (6 plataformas)	**~1.154 partidas/hora**
Pedidos por partida	**16,8 promedio (1 partida + ~16 perfiles)**
Reproceso de plata (sin red)	**~13 min**
DAG completo con bronce en disco	**15 min**
Calidad de código	**10/10 tests · ruff limpio · parse limpio**

El sesgo, en una línea por servidor

la1 (LAN)	**elo medio 3.231 — Grandmaster 39 %**
br1 (BR)	**elo medio 3.036 — Diamond 49 %**
euw1 (EUW)	**elo medio 2.585 — Emerald 48 %**
kr (KR)	**elo medio 2.254 — bimodal (Silver/Bronze + Challenger)**
la2 (LAS)	**elo medio 2.178 — Gold 37 %**
na1 (NA)	**elo medio 1.627 — Silver 33 %, Bronze 29 %**

LAS TRES FRASES DE CIERRE, SI HAY QUE RESUMIR TODO EN DIEZ SEGUNDOS

1. Una fila es una partida clasificatoria de la Season 16; el objetivo es `game_duration_s`; la clave es `match_id` y no repite.

2. El dato crudo está en disco: podemos rehacer el dataset entero sin volver a pedirle nada a Riot.

3. Sabemos dónde el dataset se aparta del ideal —no es del todo tidy y tiene un sesgo por tier— y las dos cosas están medidas y escritas, no descubiertas en esta mesa.

FF-15 · Proyecto Integrador de Ciencia de Datos · UTN FRM · 2026 · Grupo 5K09-02
Guión de la defensa de la Entrega 1. Los números salen de la corrida del 6/9/2026 y del reporte `include/output/reports/calidad.md`. El desarrollo completo de cada tema está en `docs/documentacion-proyecto.pdf`.
Fuente editable: `docs/guion-defensa-entrega1.html` · Regenerar el PDF: `python docs/generar_pdf.py guion-defensa-entrega1.html`